

# 核心玩法与算法说明

项目：五子棋（Gomoku） • Godot 4 图形化对战程序  
版本：v1.3.0 | 引擎：Godot 4.7.2-stable（兼容 4.5+） | 语言：GDScript

## 一、游戏概述

一款基于 Godot 4 的五子棋人机对战程序。玩家通过鼠标点击棋盘交叉点落子，与内置的 Alpha-Beta 剪枝 AI 对弈。**棋盘大小、连珠数、AI 难度、执子先后手**均可在右侧面板实时调整，同一套程序可覆盖从「9 路 5 连快棋」到「19 路 6 连长局」的多种玩法。

## 二、玩法规则

### 2.1 棋盘

| 项    | 取值                                |
|------|-----------------------------------|
| 棋盘规格 | 9 / 11 / 13 / 15 / 17 / 19 路（仅奇数） |
| 交叉点数 | 边长 × 边长（81 ~ 361 点）               |
| 默认规格 | 15 路                              |
| 星位   | 13 路及以上：天元 + 4 星位；11 路及以下：仅天元     |

**为什么只保留奇数边长：**偶数棋盘没有唯一的中心交叉点，天元位置、星位对称性与先后手公平性都会错位。因此滑动条步长固定为 2，且 `set_rules()` 内部对偶数一律向上取到相邻奇数，即使绕过 UI 直接调 API 也无法产生偶数盘。

### 2.2 连珠与胜负

| 项    | 取值                                                 |
|------|----------------------------------------------------|
| 连珠数  | 5 连 / 6 连（下限固定为 5）                                 |
| 胜负判定 | 横、竖、主对角线、副对角线四个方向任一方向连成 <code>win_count</code> 子即胜 |
| 长连   | 6 子及以上同样判胜                                         |

|     |           |
|-----|-----------|
| 和棋  | 棋盘下满且无人连珠 |
| 先行方 | 黑棋永远先行    |

原设计允许 3~6 连，经评审后收紧为不存在「五珠以下」的玩法：滑块下限、UI 夹取、`set_rules()` 夹取三层同时限制。

## 2.3 执子与先后手

| 选项      | 含义           |
|---------|--------------|
| 执黑 • 先行 | 玩家执黑先手（默认）   |
| 执白 • 后行 | 玩家执白，AI 执黑先手 |

## 2.4 操作方式

| 操作   | 说明                           |
|------|------------------------------|
| 落子   | 鼠标左键点击棋盘交叉点，自动吸附到最近交叉点（半格容差） |
| 悬停预览 | 鼠标停在空交叉点时显示半透明幽灵棋子，提示落点与颜色   |
| 重新开始 | 清空棋盘，按当前规则重开一局               |
| 悔棋   | 撤销到玩家回合（最多一次退一整轮）            |
| 玩法设置 | 右侧面板：棋盘大小 / 连珠数 / AI 难度 / 执子 |

## 三、AI 算法详解

AI 模块（`scripts/gomoku_ai.gd`）是一个纯算法文件：继承 `RefCounted`，不访问场景树、不引用其它脚本、不修改调用方传入的棋盘，可独立复用与单元测试。

### 3.1 总体流程

```
get_best_move(board, ai_color)
├─ 1. 载入棋盘 → 内部带哨兵边界的一维数组 _grid
│   并在载入时全量重建一次线型评分
├─ 2. 空盘 → 直接返回天元（最快的开局）
```

- └ 3. 战术预检（短路，命中则跳过搜索）
  - ① 我方一步成连珠 → 直接取胜
  - ② 对方一步成连珠 → 必须封堵
  - ③ 我方一步成活四 → 对手挡不住，直接走
- └ 4. 生成根结点候选（按威胁值排序，取前 18 个）
- └ 5. 简单难度：10% 概率随机走一个候选点
- └ 6. 迭代加深 1 → max\_depth + Alpha-Beta
  - 每层记录最优着法；超时或找到必胜着法即停

## 3.2 局面表示与增量式线型评估

这是整套算法性能的核心。

- 棋盘以带哨兵边界的一维数组 `_grid` 表示，四周填充 `WALL` 值，扫描时无需任何边界判断，且必然终止。
- 棋盘在载入时被切成若干条「线」：4 个方向上长度  $\geq 5$  的极大连续线段。每条线维护黑方分数与白方分数。
- 落子只重算经过该点的 4 条线（`_make_move` / `_unmake_move` 配合整数撤销栈），因此叶子节点评估是  $O(1)$  —— 直接返回预先维护好的全局总分之差。

评估值 = 黑方全局总分 - 白方全局总分

再按 Negamax 约定取当前行棋方的视角。

若采用「每到一个叶子就全盘重扫」的朴素做法，深度 5 在 15 路上无法在 2 秒内完成。增量维护把单次评估从  $O(n^2)$  降到  $O(1)$ ，是能搜到深度 5 的关键。

## 3.3 棋型识别与打分

棋型分值（可在 `gomoku_ai.gd` 顶部常量区调整）：

| 棋型                             | 常量                                | 分值             |
|--------------------------------|-----------------------------------|----------------|
| 连珠（达成 <code>win_count</code> ） | <code>SCORE_FIVE</code>           | 1,000,000      |
| 活四（两端皆空）                       | <code>SCORE_OPEN_FOUR</code>      | 100,000        |
| 冲四（一端被堵）                       | <code>SCORE_FOUR</code>           | 10,000         |
| 活三（两端皆空）                       | 动态，见 3.4                          | 1,000 ~ 50,000 |
| 眠三（一端被堵）                       | <code>SCORE_SLEEPING_THREE</code> | 100            |
| 活二                             | <code>SCORE_OPEN_TWO</code>       | 10             |

打分函数 `_pattern_score(count, open_a, open_b, has_gap)` 同时支持跳型：

- `xx_xx`（中间隔一个空位）→ 计为冲四（补上空位即成连珠）
- `x_xx` / `xx_x` → 计为活三

### 3.4 规则自适应（本项目的算法亮点）

AI 不是「三档固定深度」的静态程序，而是根据实际规则动态调整搜索策略与评估权重。

#### ① 棋盘尺寸 → 搜索深度

大棋盘分支因子急剧膨胀，必须收敛深度；小棋盘则可以搜得更深：

```
base = 5 (n ≤ 9)
      = 4 (n ≤ 12)
      = 3 (n ≤ 15)
      = 2 (n > 15)

depth = clampi(base + difficulty - 1, 1, 6)
      difficulty: 简单 = -1 / 普通 = 0 / 困难 = +1
```

| 棋盘    | 普通档深度 | 实测耗时   |
|-------|-------|--------|
| 9×9   | 5     | 465 ms |
| 11×11 | 4     | 140 ms |
| 13×13 | 3     | 28 ms  |
| 15×15 | 3     | 28 ms  |
| 19×19 | 2     | 9 ms   |

#### ② 连珠数 → 评估权重

所有棋型判定都相对于当前连珠数 `w`，而不是写死的 5。连子数越少，「活三 → 活四 → 取胜」的链条越短，活三的威胁就越大：

```
_pattern_score 中的判定：
count >= w      → 连珠          (SCORE_FIVE)
count == w - 1  → 活四 / 冲四
count == w - 2  → 活三（动态权重）/ 眠三
count == w - 3  → 活二
```

活三权重动态提升：

| 连珠数 | 活三分值 | 设计理由 |
|-----|------|------|
|-----|------|------|

|        |        |                                                           |
|--------|--------|-----------------------------------------------------------|
| 3 连    | 50,000 | 活三几乎等同胜势                                                  |
| 4 连    | 10,000 | 提升到与冲四同级 —— 4 连规则下活三只需两步就能赢，若仍按标准五子棋的 1000 分处理，AI 会严重低估威胁 |
| 5 连及以上 | 1,000  | 标准五子棋权重                                                   |

### ③ 边界安全

候选点生成、成五点搜索、活四点搜索三处循环都带显式守卫：

```
for r in _n:
    if r < 0 or r >= _n: continue
    var base: int = (r + PAD) * _w + PAD
    for c in _n:
        if c < 0 or c >= _n: continue
        var idx: int = base + c
        if idx < 0 or idx >= cell_count: continue
```

底层棋盘本身带 `PAD = 2` 的哨兵边界，扫描永不越界；这些守卫是针对「尺寸可变」新增的第二道保险。

## 3.5 Negamax + Alpha-Beta 剪枝

采用负极大值（Negamax）形式统一双方视角：

```
所有叶子分都取「当前行棋方视角」
父节点分数 = -子节点分数
```

配合 Alpha-Beta 剪枝，在最优着法优先搜索的前提下可剪掉大量分支。搜索过程中一旦发现必胜着法（`best > WIN_THRESHOLD`）立即提前返回。

## 3.6 迭代加深与时间控制

- 从深度 1 逐层加深到 `max_depth`
- 硬超时 2 秒：每 64 个节点检查一次时钟，每个根着法前再查一次
- 超时后保留已完成迭代的最优着法，保证任何情况下都能返回一个合法着法
- 每层把上一层的最优着法提到候选首位（PV 优先），显著提升剪枝效率
- 找到必胜着法时提前终止加深

### 3.7 Zobrist 置换表

- 用固定种子的随机数表对每个格点、每种颜色建立 Zobrist 键，增量维护局面哈希
- 哈希键按当前行棋方异或区分
- 每个表项打包进一个 64 位整数：[分数+偏移 : 高位][深度 : 4bit][标志 : 2bit][着法 : 13bit]
- 标志位区分 精确值 / 下界 / 上界，实现 Alpha-Beta 中的剪枝复用
- 含 mate-ply 修正：必胜/必败分按层数还原，避免不同深度间的分数串味
- 容量上限 20 万条，超限整体清空，防止内存无限增长

### 3.8 候选点生成与着法排序

- 只考虑距任一已有棋子切比雪夫距离  $\leq 2$  的空点（邻域计数增量维护，查询  $O(1)$ ）
- 用  $\text{PointScore} = \text{我方棋型分} \times 2 + \text{对方棋型分}$  作为启发值 —— 兼顾进攻与防守，优先搜索「既增强自己又限制对手」的点
- 根结点保留前 18 个候选，内部结点保留前 12 个
- 置换表着法提到最前

### 3.9 战术预检（短路优化）

在进入搜索前先做三次廉价扫描，命中即直接返回，完全不进入搜索树：

| 优先级 | 检查                  | 行为                |
|-----|---------------------|-------------------|
| ①   | 我方一步能连成 win_count 子 | 直接取胜              |
| ②   | 对方一步能连成 win_count 子 | 必须封堵（多点时取防守价值最高者） |
| ③   | 我方一步能形成活四           | 对手无法同时封堵两端，直接走    |

实测这一层非常有效：在  $9 \times 9 / 3$  连的一局中，AI 有多次以 1 ms、深度 0 直接算出必胜手，完全没有进入搜索树。

### 3.10 难度设计

| 档位 | 行为                                |
|----|-----------------------------------|
| 简单 | 深度基准 -1，并有 10% 概率随机走一个候选点（保留失误空间） |
| 普通 | 深度基准                              |
| 困难 | 深度基准 +1，共享 2 秒硬超时                 |

## 四、性能实测

### 4.1 规则自适应扫描（7 组规格 × 11 项断言，全部通过）

| 棋盘 / 连珠     | 格宽      | 棋子半径    | AI 深度 | AI 耗时      |
|-------------|---------|---------|-------|------------|
| 9×9 / 4 连   | 70.0 px | 28.0 px | 5     | 465 ms     |
| 9×9 / 3 连   | 70.0 px | 28.0 px | 5     | 0 ms（战术预检） |
| 11×11 / 6 连 | 56.0 px | 22.4 px | 4     | 140 ms     |
| 13×13 / 5 连 | 46.7 px | 18.7 px | 3     | 28 ms      |
| 15×15 / 5 连 | 40.0 px | 16.0 px | 3     | 28 ms      |
| 19×19 / 5 连 | 31.1 px | 12.4 px | 2     | 9 ms       |
| 19×19 / 4 连 | 31.1 px | 12.4 px | 2     | 1 ms       |

### 4.2 自对弈压力测试

| 档位       | 手数   | 单步最大耗时         | 平均耗时   | 结果          |
|----------|------|----------------|--------|-------------|
| 普通（15 路） | 18 手 | 65 ms          | 27 ms  | 无非法着法 / 无崩溃 |
| 困难（15 路） | 25 手 | <b>1537 ms</b> | 661 ms | 守住 2 秒时限    |

## 五、算法参数速查表

| 参数                   | 值         | 说明           |
|----------------------|-----------|--------------|
| SCORE_FIVE           | 1,000,000 | 连珠           |
| SCORE_OPEN_FOUR      | 100,000   | 活四           |
| SCORE_FOUR           | 10,000    | 冲四           |
| SCORE_OPEN_THREE     | 1,000     | 活三（5 连及以上基准） |
| SCORE_SLEEPING_THREE | 100       | 眠三           |
| SCORE_OPEN_TWO       | 10        | 活二           |
| TIME_LIMIT_MS        | 2,000     | 硬超时          |
| TIME_CHECK_MASK      | 63        | 每 64 节点查一次时钟 |
| TT_MAX_ENTRIES       | 200,000   | 置换表容量上限      |
| NEIGHBOR_RANGE       | 2         | 候选点邻域半径      |

|                     |      |           |
|---------------------|------|-----------|
| MAX_CANDIDATES_ROOT | 18   | 根结点候选数    |
| MAX_CANDIDATES      | 12   | 内部结点候选数   |
| EASY_RANDOM_CHANCE  | 0.10 | 简单档随机落子概率 |
| PAD                 | 2    | 哨兵边界厚度    |

## 六、接口契约

AI 模块对外只暴露 4 个方法，方便替换与测试：

```
func get_best_move(board: Array, ai_color: int) -> Vector2i
    # board: board[row][col] 的二维数组，0=空 1=黑 2=白（不会被修改）
    # 返回 (row, col)；无合法着法时返回 (-1, -1)

func set_difficulty(level: int) -> void    # 0=简单 1=普通 2=困难
func set_rules(win_count: int) -> void    # 连珠数：棋盘尺寸从 board.size() 自动推断
func reset() -> void                      # 清空置换表（两局之间调用）
```

**约束：**纯算法文件 —— 不继承 **Node**、不访问场景树、不引用其它脚本、不修改调用方传入的棋盘。